

# Pixs CRM — Referencia de API REST

Backend Express + JWT sobre SQLite. Documento de integración para consumo por bot / tool-calling.

Versión: 1.1 · Generado: 2026-07-21 · Base: Express 5 + JWT · DB: SQLite · Entorno: producción (VPS activo)

 **Acceso de producción (VPS activo):**  
**URL base:** http://149.50.152.218/crmpixs/api  
**Usuario:** admin@pixs.com · **Contraseña:** fSaWaDuT3Dq1  
Este es el entorno en uso hoy, con los datos reales del CRM cargados. Manejar estas credenciales con confidencialidad.

## 1. Información general

| Concepto                        | Valor                                                                             |
|---------------------------------|-----------------------------------------------------------------------------------|
| URL base de la API              | http://149.50.152.218/crmpixs/api                                                 |
| Prefijo de rutas (bajo la base) | /crmpixs/api (nginx → proceso interno)                                            |
| Endpoint de login               | POST http://149.50.152.218/crmpixs/api/auth/login                                 |
| Formato                         | JSON ( Content-Type: application/json ). Subida de archivos = multipart/form-data |
| Autenticación                   | JWT Bearer en header Authorization: Bearer <token>                                |
| Vigencia del token              | 7 días                                                                            |

**Modelo de auth:** el login valida *email existente + contraseña compartida* y devuelve un JWT. Todas las rutas salvo /auth/\* exigen el token en el header Authorization . En todos los ejemplos, /api/... corresponde a la ruta pública  
http://149.50.152.218/crmpixs/api/... .

## 2. Flujo de autenticación

El bot debe: (1) hacer login una vez para obtener el token, (2) guardarlo, (3) enviarlo en cada request como header Authorization . El token dura 7 días; al vencer, se repite el login.

```
# 1) Login → obtener token
curl -X POST http://149.50.152.218/crmpixs/api/auth/login \
  -H "Content-Type: application/json" \
  -d '{"email":"admin@pixs.com","password":"fSaWaDuT3Dq1"}'

# Respuesta:
# { "token": "eyJhbGciOi...", "user": { "id": "uuid", "email": "admin@pixs.com" } }

# 2) Usar el token en cualquier endpoint protegido
curl http://149.50.152.218/crmpixs/api/contacts \
  -H "Authorization: Bearer eyJhbGciOi..."

# 3) Ejemplo: resumen del CRM (datos reales cargados)
curl http://149.50.152.218/crmpixs/api/dashboard/summary \
  -H "Authorization: Bearer eyJhbGciOi..."
```

## 3. Enums / valores permitidos

| Contexto              | Valores válidos                           |
|-----------------------|-------------------------------------------|
| Estado de oportunidad | nuevo , en_proceso , ganado , perdido     |
| Estado de proyecto    | activo , pausado , finalizado , cancelado |

|                                  |                                                 |
|----------------------------------|-------------------------------------------------|
| Columna de tarea (estado)        | backlog , en_curso , revision , hecho           |
| Color de tarea                   | red , amber , green , blue , violet , pink      |
| Tipo de credencial               | servidor , base , servicio , web , email , otro |
| Tipo de info técnica de proyecto | repo , dominio , deploy , doc , link            |
| Entidad de documento             | project , contact , transaction                 |
| Entidad de actividad             | contact , opportunity , project , task          |
| Tipo de transacción              | ingreso , gasto                                 |
| Entorno de base de datos         | prod , staging , dev                            |
| Moneda (default)                 | ARS                                             |

## 4. Endpoints

### Salud & Auth sin token

**GET** /api/health

Probe de salud. Devuelve { ok: true, service: "pixs-api" }.

**POST** /api/auth/login

Body: { email, password }. Devuelve { token, user }. 401 si inválido.

**POST** /api/auth/demo-login

Login rápido con las credenciales demo del entorno. Devuelve { token, user }.

**GET** /api/auth/me

Requiere token. Devuelve { id, email, nombre, rol } del usuario actual.

### Contactos token

**GET** /api/contacts?search=

Lista contactos. search opcional (busca por LIKE).

**GET** /api/contacts/:id

Detalle de un contacto. 404 si no existe.

**GET** /api/contacts/:id/opportunities

Oportunidades del contacto.

**POST** /api/contacts

Crea contacto. Body: { nombre\* , personaContacto, email, telefono, sitioWeb, notas }. Devuelve { id }.

**PATCH** /api/contacts/:id

Actualiza contacto (mismo body que POST).

**DELETE** /api/contacts/:id

Elimina contacto.

**GET** /api/contacts/:id/people

Personas de contacto asociadas.

**POST** /api/contacts/:id/people

Agrega persona. Body: { nombre\*, puesto, email, telefono }.

**DELETE** /api/contacts/people/:personId

Elimina una persona de contacto.

## Oportunidades token

**GET** /api/opportunities

Lista todas las oportunidades.

**GET** /api/opportunities/:id

Detalle de una oportunidad.

**POST** /api/opportunities

Crea. Body: { contactId\*(uuid), titulo\*, valorEstimado, probabilidad(0-100), moneda }. Devuelve { id }.

**PATCH** /api/opportunities/:id

Actualiza (mismo body que POST).

**DELETE** /api/opportunities/:id

Elimina oportunidad.

**POST** /api/opportunities/:id/move

Cambia estado en el pipeline. Body: { estado\*, motivoPerdida }. Al pasar a ganado se crea el proyecto automáticamente.

## Proyectos token

**GET** /api/projects

Lista proyectos.

**GET** /api/projects/:id

Detalle de proyecto.

**PATCH** /api/projects/:id/state

Cambia estado. Body: { estado\* } (activo|pausado|finalizado|cancelado).

**GET** /api/projects/:id/tech-info

Info técnica del proyecto (repos, dominios, deploys, docs, links).

**POST** /api/projects/:id/tech-info

Agrega item. Body: { tipo\*, label\*, valor\* }.

**DELETE** /api/projects/tech-info/:techId

Elimina un ítem de info técnica.

## Tareas (Kanban) token

**GET** /api/tasks

Lista todas las tareas (con su proyecto).

**GET** /api/tasks/by-project/:projectId

Tareas de un proyecto.

**POST** /api/tasks

Crea. Body: { projectId\*(uuid), titulo\*, descripcion, color, estado(=backlog), asignados[uuid], createdAt, venceAt }.  
Fechas en formato YYYY-MM-DD.

**PATCH** /api/tasks/:id

Actualiza tarea completa (mismo body; estado obligatorio).

**POST** /api/tasks/:id/status

Cambia solo el estado. Body: { estado\* }.

**POST** /api/tasks/:id/move

Alias de /status. Body: { estado\* }.

**DELETE** /api/tasks/:id

Elimina tarea.

## Dinero / Finanzas token

**GET** /api/money/receivables

Cuentas por cobrar (cuotas pendientes).

**GET** /api/money/summary

Resumen financiero (ingresos, gastos, balance).

**GET** /api/money/budget/:projectId

Presupuesto y cuotas de un proyecto.

**GET** /api/money/transactions?from=&to=

Transacciones filtradas por rango de fechas (opcional).

**POST** /api/money/budgets

Crea presupuesto + genera cuotas. Body: { projectId\*, montoTotal\*, moneda(=ARS), cuotas\*(1-60), primerVencimiento\*, descripcion }.

**POST** /api/money/installments/:id/toggle

Marca cuota pagada/pendiente. Body: { projectId\*, pagar\*(bool) }. Genera/borra la transacción de cobro.

**POST** /api/money/transactions

Crea transacción. multipart/form-data con campos { tipo\*(ingreso|gasto), monto\*, moneda, categoria, fecha\*, descripcion, projectId, realizadoPor } + archivo opcional comprobante.

**POST** /api/money/transactions/:id/reintegro

Marca reintegro de un gasto. Body: { devuelto\*(bool) }.

## Documentos token

**GET** /api/documents?entityType=&entityId=

Lista documentos de una entidad. entityType : project|contact|transaction.

**POST** /api/documents

Sube archivo. multipart/form-data : file + { entityType, entityId }. Máx 20 MB.

**GET** /api/documents/:id

Descarga el binario (inline). ?download=1 fuerza descarga.

**DELETE** /api/documents/:id

Elimina documento (archivo + registro).

## Credenciales token

**Seguridad:** el listado **nunca** devuelve el secreto. El único modo de obtenerlo es el endpoint /reveal , que lo descifra con ENCRYPTION\_KEY y queda auditado.

**GET** /api/credentials?q=&projectId=&tipo=

Lista credenciales (sin secreto). Filtros opcionales.

**POST** /api/credentials

Crea. Body: { titulo\*, tipo\*, usuario, secreto, url, projectId, notas }. El secreto se cifra.

**PATCH** /api/credentials/:id

Actualiza (secreto vacío = no se toca).

**DELETE** /api/credentials/:id

Elimina credencial.

**POST** /api/credentials/:id/reveal

Descifra y devuelve { value }. Único acceso al secreto.

## Infraestructura token

**GET** /api/infra/servers

Lista servidores/VPS.

**GET** /api/infra/servers/:id

Detalle de servidor.

**POST** /api/infra/servers

Crea. Body: { nombre\*, proveedor, ipHostname, specs, os, costoMensual, descripcion }.

**GET** /api/infra/databases

Lista bases de datos.

**POST** /api/infra/databases

Crea. Body: { nombre\*, motor(=postgres), entorno(prod|staging|dev), serverId, host, puerto, credencialRef, descripcion }.

## Captación / Scraping token

**GET** /api/scraping/campaigns

Lista campañas de scraping.

**GET** /api/scraping/leads?campaignId=

Lista leads (filtrable por campaña).

**GET** /api/scraping/leads/stats

Estadísticas de leads.

**POST** /api/scraping/campaigns

Crea campaña. Body: { nombre\*, query\*, ubicacion, cantidad(1-20, =20) }.

**POST** /api/scraping/campaigns/:id/run

Ejecuta la campaña (Google Places → leads).

**POST** /api/scraping/leads/:id/enrich

Enriquece el lead con IA (Claude).

**POST** /api/scraping/leads/:id/approve

Aprueba lead → crea contacto + oportunidad.

**POST** /api/scraping/leads/:id/discard

Descarta el lead.

**DELETE** /api/scraping/leads/:id

Elimina lead.

## Actividades / Timeline token

**GET** /api/activities?entityType=&entityId=

Timeline de una entidad. entityType : contact|opportunity|project|task.

**POST** /api/activities/note

Agrega nota. Body: { entityType\*, entityId\*, contenido\* }.

## Dashboard & Usuarios token

**GET** /api/dashboard/summary

Resumen del home en un solo request: { counts, tasks, reimbursements, pipeline, cold, receivables }.

**GET** /api/users

Lista usuarios (para selectores de responsable / quién pagó).

## 5. Manejo de errores

| Código | Significado                                                                        |
|--------|------------------------------------------------------------------------------------|
| 400    | Validación Zod fallida o archivo > 20 MB. Body: { error: "mensaje" }               |
| 401    | Falta token, token inválido/expirado, o credenciales de login incorrectas          |
| 404    | Recurso no encontrado o ruta /api desconocida                                      |
| 500    | Error interno (incluye fallo al descifrar credencial si cambió la ENCRYPTION_KEY ) |

Todas las respuestas de error tienen la forma { "error": "descripción legible" }.